

Lab 4: Coalesced Memory Access and GPU Memory Types
Advanced Computerarchitecture

(B-KUL-JLI1QS)

Reinout Annaert, Michiel Van Nieuwenhove

March 15, 2024



Session Date:	4/03/2024
Class:	MElsw
Instructor:	Alex Marinšek

In this lab session, we delve into essential CUDA concepts focusing on Coalesced Memory Access and GPU memory types. Coalesced Memory Access plays a pivotal role in optimizing data retrieval by consolidating multiple memory transactions. Throughout our exploration, we will navigate the intricacies of GPU memory types, specifically Global Memory, Shared Memory and Constant Memory. The objective of this session is to provide a formal understanding of these concepts through practical examples, contributing to a comprehensive knowledge of CUDA programming principles. This report details our findings and insights gained during the two experimental exercises.

1 Part 1: Coalesced Memory Access (Both Kernels Coalesced)

In Part 1 of this exercise, the objective is to design a kernel that computes the average grayscale of an image. To accomplish this, we will load an image using the code from the previous session. The approach involves organizing data into two arrays: one arranged as `RGBRGB...RGB`, and the other as `RR...RGG...GBB...B`. Two kernels will be devised, each tailored for processing one of the aforementioned arrays. Each pixel's grayscale value will be derived by a single thread, iterating over the three RGB values. For the first kernel, threads are assigned in such a way that each thread handles adjacent RGB pairs, ensuring that consecutive threads access consecutive memory locations (RGB pairs) in the input image array. For the second one, each thread is assigned to process three values that are located at intervals of width x height in the input image array, representing the same channel. Since threads access values with indices spaced apart, they are not accessing consecutive memory locations in the input image array.

Both kernels employ coalesced memory access for performing grayscale conversion. In Figure 1a, a discrepancy in performance is apparent due to thread divergence. However, upon introducing redundant calculations in both kernels to align their computational loads, as depicted in Figure 1b, the execution times converge. Consequently, it can be inferred that there is no inherent disparity between sorting images by pixel or color for this operation. Nonetheless, slight deviations persist when fewer threads are utilized. This discrepancy may stem from the kernel that sorts by pixels incurring a dual incrementation operation, which was not accounted for as a latency in the alternative kernel. While this operation is not computationally intensive, its frequency escalates with fewer threads due to striding. Consequently, the operation is invoked more frequently under these conditions compared to when numerous threads are employed.

2 Part 2: Global, Shared and Constant Memory

In Part 2 of our exploration, the focus shifts towards an in-depth analysis of memory types, specifically Global, Shared, and Constant memory, through the implementation of a matrix multiplication kernel. The objective is to assess the processing time for kernels utilizing different memory configurations. The provided code presents three distinct kernels: `matrixMulGlobal` utilizing global memory only, `matrixMulGlobalShared` employing both global and shared memory, and `matrixMulGlobalConstant` making use of global and constant memory.

The matrix multiplication operation is conducted on randomly generated matrices, and the code measures the execution time for each kernel in the context of varying array sizes. The `main` function systematically tests the kernels across different array sizes, recording the execution times for global-only, global and shared, and global and constant memory configurations. For each array

size and memory configuration, 1000 measurements were conducted, and the average execution time was computed from these measurements.

As the array size increases, the shared memory configuration consistently outperforms both the global-only and global-constant memory configurations. Notably, the execution times for the global-only and global-shared memory configurations exhibit relative stability, whereas the Constant Memory configuration shows an upward trend as the array size grows. This trend is visually represented in Figure 2.

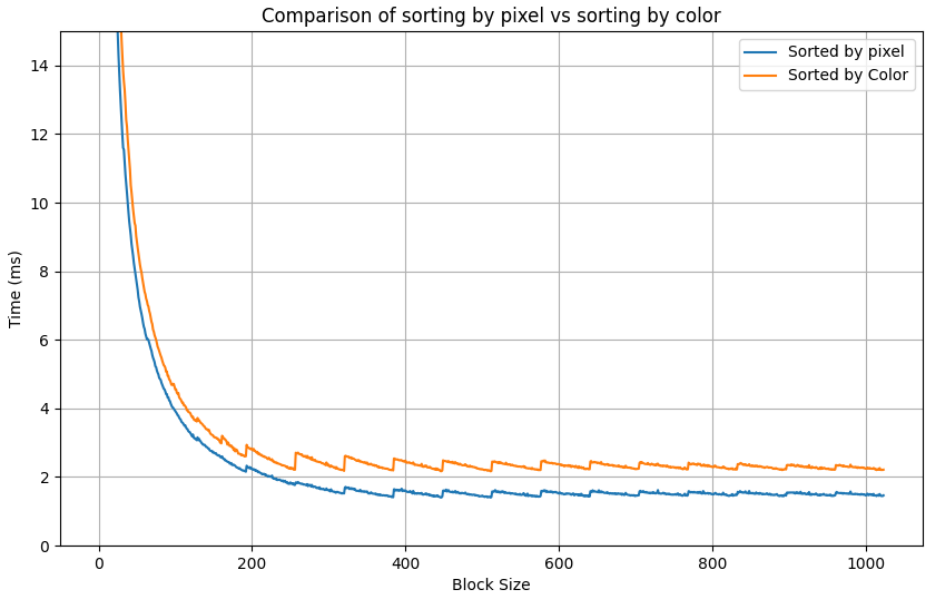
The observed performance trends in matrix multiplication can be attributed to the specific characteristics of each memory configuration. The shared memory configuration excels as the array size increases due to its ability to efficiently share data among threads within the same block. This cooperative data sharing minimizes global memory access, resulting in improved overall performance. The relatively constant execution times for the global-only and global-shared memory configurations suggest that, in the context of matrix multiplication, the utilization of shared memory does not provide a significant advantage for smaller array sizes. These configurations likely benefit from the inherent parallelism of GPU architecture but may not fully exploit the advantages of shared memory until the computational workload becomes more substantial.

On the other hand, the Constant Memory configuration experiences an increase in execution times as the array size grows. The performance of the Constant Memory configuration diminishes as the array size becomes too large for the available constant memory, causing an increase in execution times. This emphasizes the importance of considering memory limitations and selecting an appropriate memory configuration based on the specific requirements of the computational task..

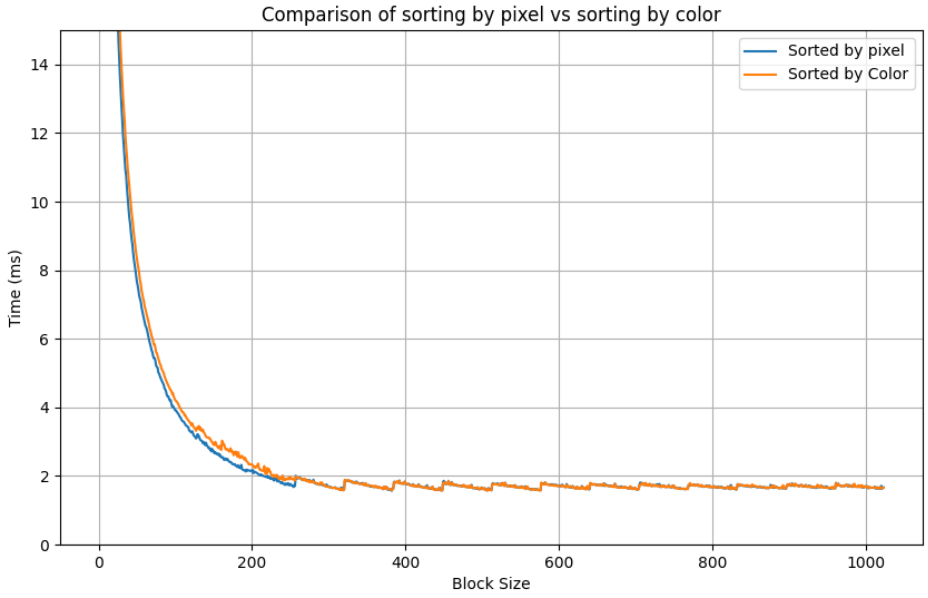
The limitation of shared memory capacity to 48 kB restricts our ability to observe trends for larger array sizes in Figure 2a. To overcome this limitation, we employed a strategy depicted in Figure 2b, where we divided the larger array into tiles. By doing so, we were able to explore the performance trends for increasing array sizes. Notably, the results indicate that the shared memory configuration consistently outperforms the global memory configuration as the array size increases.

In conclusion, the efficiency of matrix multiplication depends on a delicate balance between memory access patterns, data sharing, and the computational workload. The shared memory configuration proves advantageous for larger array sizes, showcasing the importance of tailoring memory configurations to the specific characteristics of the computational task at hand.

The GitHub repository containing the source code can be found at *Github Repository*



(a) No added latencies

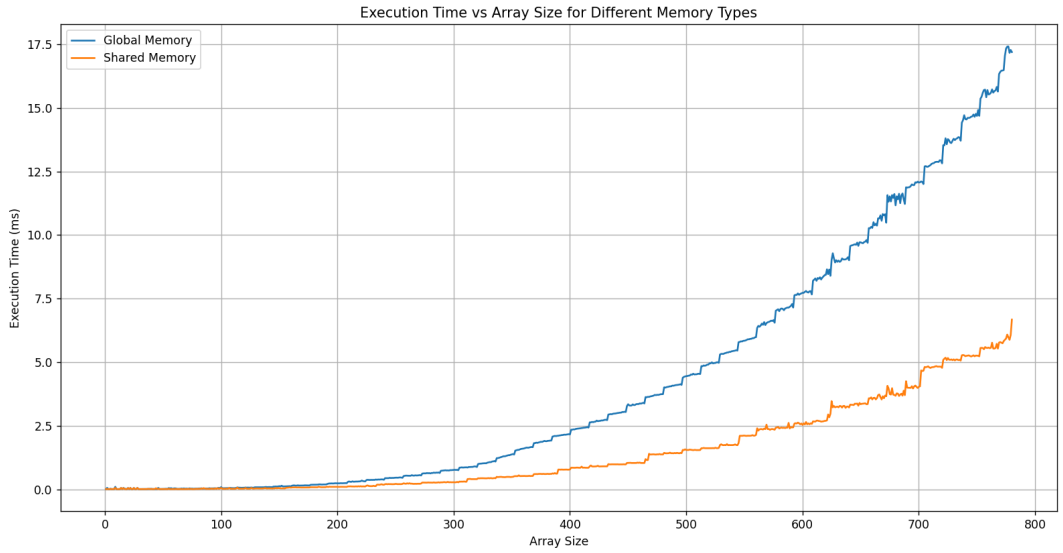


(b) Added latencies

Figure 1: Execution Times for sorting by pixel vs sorting by color. The second one has added redundant calculations to encompass for the thread divergence (Part 1)



(a) 1 Tile



(b) Multiple Tiles

Figure 2: Execution Times for Different Memory Types (Part 2)